

Hash Table Implementation and Performance Analysis

1. Implementation Details

The hash table was implemented using an array-based structure in Python using a list. Each index in the array represents a bucket that stores key-value pairs. Lists are used inside each bucket to store multiple elements when collisions occur.

Two hashing techniques were implemented the division method and the multiplication method. These methods compute the index where a key should be stored in the table.

To handle collisions this implementation uses separate chaining where multiple keys that hash to the same index are stored in a list at that index.

The hash table supports three main operations:

- `insert(key, value)` to add key-value pairs
- `search(key)` to retrieve a value associated with a key
- `delete(key)` to remove a key-value pair from the table

I did not implement dynamic resizing as chaining allows the table to hold more elements than the table size although large load factors may increase chain lengths.

These were the results I got after running

```
Table Size: 101 Load Factor: 0.5 Distribution: uniform Successful Search Time: 3.422800000407733e-07  
Unsuccessful Search Time: 2.761799987638369e-07 Avg Chain: 0.49504950495049505 Max Chain: 3
```

Table Size: 101 Load Factor: 0.5 Distribution: skewed Successful Search Time: 2.858800007743412e-07
Unsuccessful Search Time: 2.583800005595549e-07 Avg Chain: 0.40594059405940597 Max Chain: 2

Table Size: 101 Load Factor: 0.5 Distribution: sequential Successful Search Time:
2.615399989736034e-07 Unsuccessful Search Time: 2.6879999950324416e-07 Avg Chain: 0.49504950495049505
Max Chain: 1

Table Size: 101 Load Factor: 0.75 Distribution: uniform Successful Search Time: 2.9398666811175645e-07
Unsuccessful Search Time: 5.508266652517099e-07 Avg Chain: 0.7425742574257426 Max Chain: 5

Table Size: 101 Load Factor: 0.75 Distribution: skewed Successful Search Time: 2.802400013024453e-07
Unsuccessful Search Time: 2.645333309677274e-07 Avg Chain: 0.6039603960396039 Max Chain: 3

Table Size: 101 Load Factor: 0.75 Distribution: sequential Successful Search Time:
2.5549333561987925e-07 Unsuccessful Search Time: 2.6334666472394017e-07 Avg Chain: 0.7425742574257426
Max Chain: 1

Table Size: 101 Load Factor: 1.0 Distribution: uniform Successful Search Time: 2.9182999924159956e-07
Unsuccessful Search Time: 3.2326000109605957e-07 Avg Chain: 1.0 Max Chain: 3

Table Size: 101 Load Factor: 1.0 Distribution: skewed Successful Search Time: 2.738600005613989e-07
Unsuccessful Search Time: 2.6256999944962446e-07 Avg Chain: 0.8217821782178217 Max Chain: 3

Table Size: 101 Load Factor: 1.0 Distribution: sequential Successful Search Time:
2.535199996600568e-07 Unsuccessful Search Time: 2.628300012474938e-07 Avg Chain: 1.0 Max Chain: 1

Table Size: 101 Load Factor: 1.5 Distribution: uniform Successful Search Time: 3.1063000051290145e-07
Unsuccessful Search Time: 2.8754999902957934e-07 Avg Chain: 1.495049504950495 Max Chain: 8

Table Size: 101 Load Factor: 1.5 Distribution: skewed Successful Search Time: 2.6933999834000135e-07
Unsuccessful Search Time: 2.6450999939697795e-07 Avg Chain: 1.0198019801980198 Max Chain: 3

Table Size: 101 Load Factor: 1.5 Distribution: sequential Successful Search Time:
2.655000002960151e-07 Unsuccessful Search Time: 2.78810000509111e-07 Avg Chain: 1.495049504950495 Max
Chain: 2

Table Size: 503 Load Factor: 0.5 Distribution: uniform Successful Search Time: 3.060499989260279e-07
Unsuccessful Search Time: 2.801699997689866e-07 Avg Chain: 0.4990059642147117 Max Chain: 5

Table Size: 503 Load Factor: 0.5 Distribution: skewed Successful Search Time: 2.6199999865639255e-07
Unsuccessful Search Time: 2.6745000013761456e-07 Avg Chain: 0.30218687872763417 Max Chain: 1

Table Size: 503 Load Factor: 0.5 Distribution: sequential Successful Search Time:
2.521100009289512e-07 Unsuccessful Search Time: 2.6427999955558334e-07 Avg Chain: 0.4990059642147117
Max Chain: 1

Table Size: 503 Load Factor: 0.75 Distribution: uniform Successful Search Time: 3.0774999913774083e-07
Unsuccessful Search Time: 6.312699997579329e-07 Avg Chain: 0.7495029821073559 Max Chain: 4

Table Size: 503 Load Factor: 0.75 Distribution: skewed Successful Search Time: 2.7065000040238375e-07
Unsuccessful Search Time: 2.64420000348764e-07 Avg Chain: 0.3717693836978131 Max Chain: 1

Table Size: 503 Load Factor: 0.75 Distribution: sequential Successful Search Time:
2.7825999950437106e-07 Unsuccessful Search Time: 2.697299987630686e-07 Avg Chain: 0.7495029821073559
Max Chain: 1

Table Size: 503 Load Factor: 1.0 Distribution: uniform Successful Search Time: 3.6681999972643096e-07
Unsuccessful Search Time: 2.9868999945392716e-07 Avg Chain: 1.0 Max Chain: 4

Table Size: 503 Load Factor: 1.0 Distribution: skewed Successful Search Time: 2.5837999828581815e-07
Unsuccessful Search Time: 2.6642999955583946e-07 Avg Chain: 0.4254473161033797 Max Chain: 2

Table Size: 503 Load Factor: 1.0 Distribution: sequential Successful Search Time:
2.8162000035081294e-07 Unsuccessful Search Time: 2.796000012494915e-07 Avg Chain: 1.0 Max Chain: 1

Table Size: 503 Load Factor: 1.5 Distribution: uniform Successful Search Time: 3.3346000009260026e-07
Unsuccessful Search Time: 3.0709000157003177e-07 Avg Chain: 1.4990059642147118 Max Chain: 8

Table Size: 503 Load Factor: 1.5 Distribution: skewed Successful Search Time: 2.7245999945080257e-07
Unsuccessful Search Time: 2.758799996627204e-07 Avg Chain: 0.5506958250497018 Max Chain: 2

Table Size: 503 Load Factor: 1.5 Distribution: sequential Successful Search Time:
3.0002999892531077e-07 Unsuccessful Search Time: 8.378500001526845e-07 Avg Chain: 1.4990059642147118
Max Chain: 2

Table Size: 1009 Load Factor: 0.5 Distribution: uniform Successful Search Time: 5.605299998023838e-07
Unsuccessful Search Time: 5.001300019102927e-07 Avg Chain: 0.49950445986124875 Max Chain: 4

Table Size: 1009 Load Factor: 0.5 Distribution: skewed Successful Search Time: 2.6999000056093793e-07
Unsuccessful Search Time: 2.6278999939677303e-07 Avg Chain: 0.2091179385530228 Max Chain: 1

Table Size: 1009 Load Factor: 0.5 Distribution: sequential Successful Search Time:
3.432299990890897e-07 Unsuccessful Search Time: 2.7599000077316307e-07 Avg Chain: 0.49950445986124875
Max Chain: 1

Table Size: 1009 Load Factor: 0.75 Distribution: uniform Successful Search Time:
3.1063000051290145e-07 Unsuccessful Search Time: 2.8658000019277095e-07 Avg Chain: 0.7492566897918731
Max Chain: 5

Table Size: 1009 Load Factor: 0.75 Distribution: skewed Successful Search Time: 2.703999984987604e-07
Unsuccessful Search Time: 2.5672999981907196e-07 Avg Chain: 0.2666005946481665 Max Chain: 1

Table Size: 1009 Load Factor: 0.75 Distribution: sequential Successful Search Time:
2.9390000008788777e-07 Unsuccessful Search Time: 2.8182999812997876e-07 Avg Chain: 0.7492566897918731
Max Chain: 1

Table Size: 1009 Load Factor: 1.0 Distribution: uniform Successful Search Time: 3.3493000046291854e-07
Unsuccessful Search Time: 3.017100016222685e-07 Avg Chain: 0.9990089197224975 Max Chain: 6

Table Size: 1009 Load Factor: 1.0 Distribution: skewed Successful Search Time: 2.708499982873036e-07
Unsuccessful Search Time: 2.7380000119592294e-07 Avg Chain: 0.2983151635282458 Max Chain: 1

Table Size: 1009 Load Factor: 1.0 Distribution: sequential Successful Search Time:
2.9653999945367104e-07 Unsuccessful Search Time: 2.8053000050931587e-07 Avg Chain: 1.0 Max Chain: 1

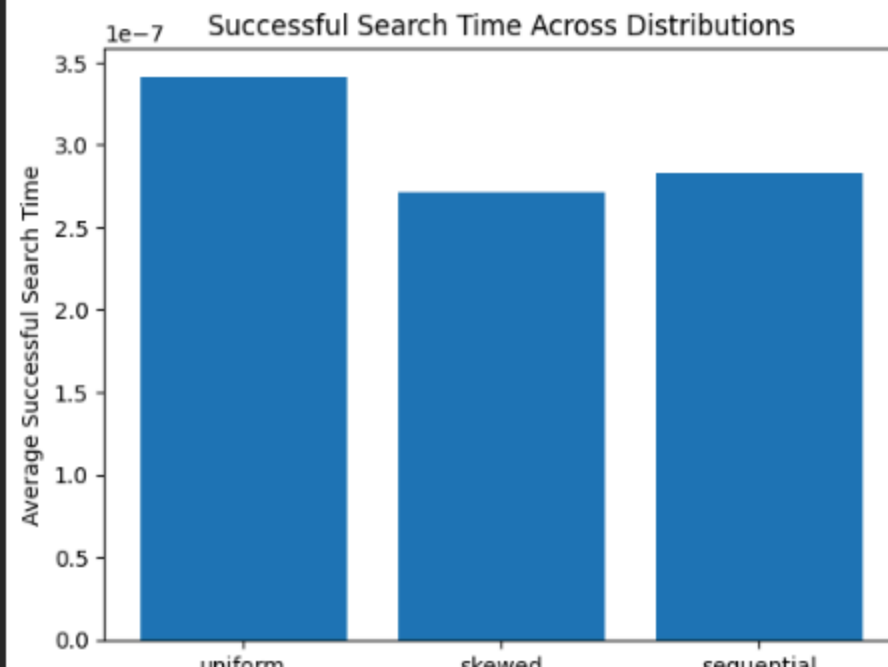
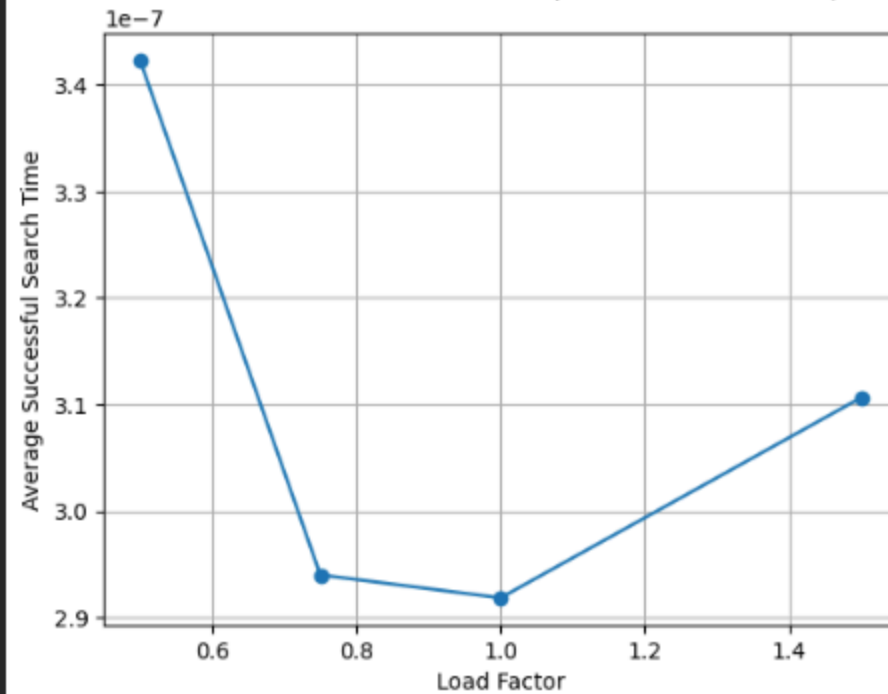
Table Size: 1009 Load Factor: 1.5 Distribution: uniform Successful Search Time: 3.3732000019881526e-07
Unsuccessful Search Time: 3.4326999866607364e-07 Avg Chain: 1.4995044598612488 Max Chain: 7

Table Size: 1009 Load Factor: 1.5 Distribution: skewed Successful Search Time: 2.7488999876368325e-07
Unsuccessful Search Time: 2.6958000034937865e-07 Avg Chain: 0.3290386521308226 Max Chain: 1

Table Size: 1009 Load Factor: 1.5 Distribution: sequential Successful Search Time:
3.146300014122971e-07 Unsuccessful Search Time: 2.996600005644723e-07 Avg Chain: 1.4995044598612488
Max Chain: 2

TABLE 5.11: 100% Load Factor, 1% Distribution, Sequential Successful Search Time, 5.1402

Load Factor vs Successful Search Time (Uniform Distribution, Size 101)



2. Performance Comparison

Load Factor vs Search Time

Experiments were conducted using table sizes 101, 503, and 1009 with load factors 0.5, 0.75, 1.0, and 1.5.

The results show that as the load factor increases the average search time slightly increases because more elements are stored in each bucket.

I also created plots to show the relationship between load factor and search time.

Key Distribution Comparison

Three key distributions were tested:

- Uniform distribution
- Skewed distribution
- Sequential keys

Search time varied slightly depending on the distribution. Uniform keys generally produced stable performance, while skewed and sequential keys sometimes created uneven distributions depending on the hash function.

A bar chart was generated to compare the search time across the different key distributions.

Memory Overhead Analysis

Since this implementation uses chaining, memory overhead comes from storing lists at each bucket.

Two metrics were measured:

- Average chain length, which shows the average number of elements per bucket
- Maximum chain length, which shows the largest bucket in the table

The results showed that the average chain length increases as the load factor increases.

3. Observations and Discussion

Performance starts to degrade when the load factor becomes larger than 1 because more elements are stored in each bucket and searches take longer.

With chaining the hash table can still function even when the table is full but longer chains may slow down search operations.

In contrast open addressing stores all elements directly in the table and resolves collisions by probing other positions. This approach uses less memory but performance degrades quickly as the table becomes full.

Key distribution also affects performance. Uniform distributions tend to spread keys evenly across the table while skewed distributions can create clusters that increase collisions.

Overall the experiments show that keeping the load factor reasonably low helps maintain efficient hash table performance.

“I usually always run my code in colab so I used Colab AI to fix mistakes in my code as I couldn't debug them myself and referred to a lot of youtube videos regarding these topics to find a suitable method to do this . “